

TD informatique

CORRECTION

1) Cette fonction retourne une grille composée de n listes de taille n contenant uniquement des zéros : que des cases saines.

```
2) def init(n):
    M = grille(n)
    i = rd.randrange(n)
    j = rd.randrange(n)
    M[i][j] = 1
    return(M)
```

3) On peut créer des variables $n0, n1, n2, n3$ initialisées à 0 et faire des tests à chaque case pour savoir qui incrémenter. Ou on peut ruser :

```
def compte(G):
    n = len(G)
    L = [0,0,0,0]
    for i in range(n):
        for j in range(n):
            L[G[i][j]] += 1
    return(L)
```

4) Elle renvoie un booléen : le résultat d'un test avec $=$.

```
5) #Ligne 12
return (G[0][j-1]-1)*(G[0][j+1]-1)*(G[1][j-1]-1)*(G[1][j]-1)*(G[1][j+1]-1)==0
#Ligne 20
return (G[i-1][j-1]-1)*(G[i-1][j]-1)*(G[i-1][j+1]-1)*(G[i][j-1]-1)*(G[i][j+1]-1)*
        (G[i+1][j-1]-1)*(G[i][j+1]-1)*(G[i+1][j+1]-1)==0
```

```
6) def suivant(G,p1,p2):
    n = len(G)
    M = grille(n) # M ne contient que des 0, sera remplie avec les valeurs de l'étape suivante de G
    for i in range(n):
        for j in range(n):
            if G[i][j] == 0 and est_exposee(G,i,j): # Cas sain et voisin infecté
                if bernouilli(p2) == 1: # test
                    M[i][j]=1 # devient infecté, sinon c'est déjà 0 dans M
            elif G[i][j] == 1: # cas infecté
                if bernouilli(p1) == 1: # test
                    M[i][j]=3 # devient décédé
            else:
                M[i][j]=2 # devient retabli
        else: M[i][j] == G[i][j] # Cas stables 2 reste 2 et 3 reste 3
    return(M)
```

```
7) def simulation(n,p1,p2):
    G = init(n)
    n0,n1,n2,n3 = compte(G)
    while n1 > 0:
        G = suivant(G)
        n0,n1,n2,n3 = compte(G)
    return([n0/n**2, n1/n**2, n2/n**2, n3/n**2])
```

8) A la fin de la simulation : $x1 = 0$ car la simulation s'arrête quand il n'y a plus d'infecté.

On a $x0 + x1 + x2 + x3 = 1$, proportion totale. On a $x_{\text{atteinte}} = x1 + x2 + x3 = 1 - x0$ tout le monde sauf les sains.

```
9) def seuil(Lp2,Lxa):
    n = len(Lp2)
    g,d = 0, n-1
    while d-g > 1:
```

```
        m=(g+d) // 2
        if Lxa[m] > 0.5:
            d = m
        else:
            g = m
    return(Lp2[g],Lp2[d])
10) def init_vac(n,q):
    G = init(n)
    nvac = int(q * n**2) # nombre à vacciner prop * pop totale
    k = 0
    while k < nvac :
        i = rd.randrange(n)
        j = rd.randrange(n)
        if G[i][j] == 0 : # il ne faut vacciner que les sains !
            G[i][j] = 2
            k += 1
    return(G)
```